Fast Flow Engine

FFENGINE

Agent Coding Architecture

AI Agent'lar İçin Kodlama, Dosya Yapısı ve Wave-Bazlı İlerleme Rehberi

TAD v6.0 + Agent Architecture v6.0 Tabanlı

Versiyon
Agent Coding Architecture v6.0
Baz Dokümanlar
FFEngine TAD v6.0 + Agent Architecture v6.0
Dönem
2026-Q1 / Q2
Hedef
AI agent'ların ffengine projesini wave bazlı kodlaması için eksiksiz rehber
Kapsam
Community (Wave 1–4) + Enterprise (Wave 5–8)

İçindekiler

00 — Agent Kodlama Prensipleri

Bu doküman, AI agent'ların ffengine projesini sıfırdan kodlayabilmesi için gereken tüm kararları, dosya yapısını ve wave ilerleme sırasını tanımlar. Her wave bağımsız olarak teslim edilebilir ve bir sonraki wave'in ön koşulunu oluşturur.

Temel Kurallar

Kural #1 — Versiyonu Önce Belirle: Her dosya, sınıf veya fonksiyon üretmeden önce şu soruyu sor: 'Bu Community mi, Enterprise mı, yoksa Ortak mı?' Cevap Wave tablosundan okunur. Yanlış versiyona atama yapılmış kod çakışmaya neden olur.

Kural #2 — Wave Sırasını Koru: Her wave, bir önceki wave'in çıktısına bağımlıdır. Wave 3'ü Wave 2 tamamlanmadan başlatma. Bağımlılık tablosu her wave başında açıkça gösterilmiştir.

Kural #3 — Test Önce: Her wave'in sonunda o wave'e ait integration test'ler çalıştırılır. Testler geçmeden bir sonraki wave'e geçilmez.

Kural #4 — Engine-Swap Korunmalı: Community Python Engine ile Enterprise C Engine aynı interface'i paylaşır. Wave 5+ sırasında bu interface bozulmamalıdır. FFEngineOperator automatic fallback her zaman çalışmalıdır.

Toplam Wave Özeti

Wave	İsim	Kapsam	Dönem	Çıktı
W1	Foundation & Core Infrastructure	Ortak	Q1 2026 — Sprint 1-2	Proje iskeleti, DBSession, BaseDialect, config yükleyici
W2	Dialect & TypeMapper	Ortak	Q1 2026 — Sprint 2-3	PostgresDialect, OracleDialect, MSSQLDialect, TypeMapper, DDLGenerator
W3	Community Engine (Python Core)	Community	Q1 2026 — Sprint 3-4	SourceReader, Streamer, TargetWriter, ETLManager (Community)
W4	Airflow Integration & ETL Studio	Ortak	Q1 2026 — Sprint 4-5	FFEngineOperator, Auto-DAG, 3-Faz Pattern, ETL Studio UI, validate_config
W5	Community GA — Operasyonel Araçlar	Community	Q1 2026 — Sprint 5-6	mapping_generator, send_email, debugpy,

Wave	İsim	Kapsam	Dönem	Çıktı
				Partitioning, Dry-Run — Community RELEASE
W6	Enterprise Queue Runtime (C Binding)	Enterprise	Q2 2026 — Sprint 7-8	FFEnvelope, IngressQueue, EgressQueue, CheckpointStore, DeliveryManager
W7	Enterprise Native Bulk API	Enterprise	Q2 2026 — Sprint 8-9	COPY TO STDOUT/FROM STDIN, BCP, OCI_BATCH, N Reader/M Writer, Adaptive Micro-Batch
W8	Enterprise GA — DLQ, Multi-Lane & Guarantee	Enterprise	Q2 2026 — Sprint 9-10	DLQ Policy, Multi-Lane Pipeline, Delivery Guarantee Matrix — Enterprise RELEASE

01 — Proje Dizin Yapısı (Tam)

Aşağıdaki dizin yapısı ffengine projesinin tamamını kapsar. Her dosyanın hangi wave'de oluşturulacağı etiketlenmiştir.

```
ffengine/
├── pyproject.toml                    # W1 — paket tanımı, bağımlılıklar
├── README.md                         # W1
├── LICENSE                           # W1
│
├── src/
│   └── ffengine/
│       ├── __init__.py               # W1
│       │
│       ├── core/                     # ── CORE ENGINE ──────────────────
│       │   ├── __init__.py           # W1
│       │   ├── config.py             # W1 — YAML yükleyici, şema doğrulama
│       │   ├── engine_interface.py   # W1 — BaseEngine abstract class
│       │   ├── etl_manager.py        # W3 — ETLManager (Community), W6
(Enterprise path)
│       │   ├── progress_tracker.py   # W3 — rows/sec, tahmini süre, XCom
metrikleri
│       │   └── error_handler.py      # W3 — hata sınıflandırma, log etiketleme
│       │
│       ├── db/                       # ── VERİTABANI KATMANI ──────────
│       │   ├── __init__.py           # W1
│       │   ├── session.py            # W1 — DBSession context manager
│       │   ├── connection_factory.py # W1 — sürücü tespiti, bağlantı üretimi
│       │   └── pool_manager.py       # W4 — keep-alive, connection pooling
│       │
│       ├── dialect/                  # ── DIALECT KATMANI (ORTAK) ──────
│       │   ├── __init__.py           # W2
│       │   ├── base.py               # W2 — BaseDialect abstract class
│       │   ├── postgres.py           # W2 — PostgresDialect
│       │   ├── oracle.py             # W2 — OracleDialect
│       │   ├── mssql.py              # W2 — MSSQLDialect
│       │   └── registry.py           # W2 — dialect seçici
│       │
│       ├── mapping/                  # ── TYPE MAPPING & DDL ──────────
│       │   ├── __init__.py           # W2
│       │   ├── type_mapper.py        # W2 — TypeMapper (Oracle/MSSQL/PG eşleme)
│       │   ├── ddl_generator.py      # W2 — DDLGenerator (CREATE TABLE üretici)
│       │   └── schema_reader.py      # W2 — MetadataInspector / SchemaReader
│       │
│       ├── pipeline/                 # ── COMMUNITY ENGINE ────────────
│       │   ├── __init__.py           # W3
│       │   ├── source_reader.py      # W3 — SourceReader (fetchmany cursor)
│       │   ├── streamer.py           # W3 — Streamer (yield generator,
backpressure)
│       │   ├── target_writer.py      # W3 — TargetWriter (executemany)
│       │   └── transformer.py        # W3 — Transformer + TABLE_RULES
│       │
│       ├── filters/                  # ── DYNAMIC FILTERS ─────────────
│       │   ├── __init__.py           # W4
│       │   ├── binding_resolver.py   # W4 — resolve_where_with_bindings()
│       │   └── partitioner.py        # W4 — tüm partition stratejileri
│       │
│       ├── airflow/                  # ── AIRFLOW ENTEGRASYONU ────────
│       │   ├── __init__.py           # W4
│       │   ├── operator.py           # W4 — FFEngineOperator
│       │   ├── dag_generator.py      # W4 — Auto-DAG Generator
│       │   ├── dag_patterns.py       # W4 — 3-Faz DAG pattern
│       │   └── etl_studio/           # W4 — ETL Studio UI (Airflow plugin)
│       │       ├── __init__.py
│       │       ├── plugin.py         # W4 — Airflow plugin kayıt
│       │       ├── views.py          # W4 — Admin > ETL Studio views
```

```
│         │         └── static/           # W4 — CSS/JS assets
│         │
│         ├── validation/                 # ── KONFİGÜRASYON DOĞRULAMA ────
│         │   ├── __init__.py             # W4
│         │   └── config_validator.py     # W4 — validate_config()
│         │
│         ├── tools/                      # ── OPERASYONEL ARAÇLAR ──────────
│         │   ├── __init__.py             # W5
│         │   ├── mapping_generator.py    # W5 — CLI: otomatik YAML mapping üretici
│         │   ├── send_email_custom.py    # W5 — on_failure_callback email bildirimi
│         │   └── debug_utils.py          # W5 — debugpy wait_for_debug()
│         │
│         └── enterprise/                 # ── ENTERPRISE ENGINE ────────────
│             ├── __init__.py             # W6
│             ├── engine.py               # W6 — C Engine Python binding / interface
│             ├── queue_runtime/          # W6 — Queue Runtime modülleri
│             │   ├── __init__.py         # W6
│             │   ├── envelope.py         # W6 — FFEnvelope
│             │   ├── ingress_queue.py    # W6 — IngressQueue
│             │   ├── egress_queue.py     # W6 — EgressQueue
│             │   ├── checkpoint_store.py # W6 — CheckpointStore
│             │   ├── delivery_manager.py # W6 — DeliveryManager
│             │   └── backpressure.py     # W6 — BackpressureController
│             ├── bulk/                   # W7 — Native Bulk API
│             │   ├── __init__.py         # W7
│             │   ├── pg_copy.py          # W7 — COPY TO/FROM STDIN (binary)
│             │   ├── mssql_bcp.py        # W7 — BCP / fast_executemany
│             │   ├── oracle_oci.py       # W7 — OCI_BATCH_ERRORS
│             │   └── worker_pool.py      # W7 — N Reader / M Writer
│             └── policy/                 # W8 — Delivery & DLQ policy
│                 ├── __init__.py         # W8
│                 ├── dlq_policy.py       # W8 — Dead Letter Queue
│                 ├── retry_policy.py     # W8 — exponential backoff
│                 ├── delivery_policy.py # W8 — at-least-once / exactly-once
(koşullu)
│                 └── multi_lane.py       # W8 — Multi-Lane Pipeline
│
├── tests/                                # ── TESTLER ──────────────────────
│   ├── unit/
│   │   ├── test_config.py                # W1
│   │   ├── test_session.py               # W1
│   │   ├── test_dialects.py              # W2
│   │   ├── test_type_mapper.py           # W2
│   │   ├── test_ddl_generator.py         # W2
│   │   ├── test_source_reader.py         # W3
│   │   ├── test_streamer.py              # W3
│   │   ├── test_target_writer.py         # W3
│   │   ├── test_etl_manager.py           # W3
│   │   ├── test_binding_resolver.py      # W4
│   │   ├── test_partitioner.py           # W4
│   │   ├── test_operator.py              # W4
│   │   ├── test_validate_config.py       # W4
│   │   ├── test_ingress_queue.py         # W6
│   │   ├── test_checkpoint_store.py      # W6
│   │   ├── test_pg_copy.py               # W7
│   │   └── test_dlq_policy.py            # W8
│   └── integration/
│       ├── test_pg_to_pg.py              # W3 — Community: PG→PG tam aktarım
│       ├── test_pg_to_mssql.py           # W2/W3
│       ├── test_pg_to_oracle.py          # W2/W3
│       ├── test_airflow_dag.py           # W4
│       ├── test_etl_studio.py            # W4
│       ├── test_community_e2e.py         # W5 — Community tam E2E
│       ├── test_enterprise_queue.py      # W6
│       ├── test_enterprise_bulk.py       # W7
│       └── test_enterprise_e2e.py        # W8 — Enterprise tam E2E
│
├── projects/                             # ── PROJE YAPISI ─────────────────
```

```
│   └── {proje_adı}/
│       └── {domain}/
│           ├── level1/
│           │   ├── src_to_stg/
│           │   │   ├── config.yaml   # W4
│           │   │   ├── mappings/     # W5 (mapping_generator çıktısı)
│           │   │   └── sql/          # W4
│           │   └── src_to_ods/
│           └── level2/
│               └── stg_to_final/
│                   ├── config.yaml
│                   └── sql/
│
├── docs/                             # ── DOKÜMANLAR ────────────────────
│   ├── TAD_v6.docx                   # Teknik Analiz Dokümanı
│   └── Agent_Architecture_v6.docx   # Bu doküman
│
└── docker/
    ├── Dockerfile                    # W1
    └── docker-compose.yml            # W1
```

02 — Sınıf Hiyerarşisi ve Interface Kontratı

ffengine'in engine-swap mimarisi için tüm abstract sınıflar ve interface kontratları aşağıda tanımlanmıştır. Agent, bu kontratları bozmadan implementasyon yapmalıdır.

BaseEngine — Engine-Swap Kontratı (W1)

```
# src/ffengine/core/engine_interface.py
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class ETLResult:
    rows: int
    duration_seconds: float
    throughput: float          # rows/sec
    partitions_completed: int
    errors: list[str]

class BaseEngine(ABC):
    """
    ffengine Engine-Swap kontratı.
    Community (Python Engine) ve Enterprise (C Engine) bu sınıfı implement eder.
    FFEngineOperator sadece bu interface'i bilir — hangi engine olduğunu bilmez.
    """
    @abstractmethod
    def run(self, config_path: str, task_group_id: str) -> ETLResult:
        """Bir ETL task'ını çalıştırır ve ETLResult döner."""
        ...

    @abstractmethod
    def is_available(self) -> bool:
        """Bu engine'in çalışma ortamında yüklü olup olmadığını kontrol eder."""
        ...

    @classmethod
    def detect(cls, preference: str = "auto") -> "BaseEngine":
        """
        engine='auto' → Enterprise C Engine yüklüyse onu, yoksa Community'yi
seçer.
        engine='community' → her zaman Community.
        engine='enterprise' → C Engine yoksa hata fırlatır.
        """
        from ffengine.enterprise.engine import CEngine        # W6'da implement
edilir
        from ffengine.pipeline.etl_manager import PythonEngine # W3'te implement
edilir

        if preference == "community":
            return PythonEngine()
        if preference == "enterprise":
            engine = CEngine()
            if not engine.is_available():
                raise RuntimeError("Enterprise C Engine yüklü değil")
            return engine
        # auto: fallback
        c_engine = CEngine()
        return c_engine if c_engine.is_available() else PythonEngine()
```

BaseDialect — Dialect Kontratı (W2)

```
# src/ffengine/dialect/base.py
from abc import ABC, abstractmethod
```

```
class BaseDialect(ABC):

    @abstractmethod
    def connect(self, params: dict):
        """DB bağlantısı açar ve döner."""
        ...

    @abstractmethod
    def create_cursor(self, conn, server_side: bool = False):
        """Server-side cursor oluşturur."""
        ...

    @abstractmethod
    def get_table_schema(self, conn, schema: str, table: str) -> list[dict]:
        """Kolon adı, tip, nullable bilgilerini döner."""
        ...

    @abstractmethod
    def generate_bulk_insert_query(self, schema: str, table: str, cols:
list[str]) -> str:
        """Community: standart INSERT | Enterprise: COPY/BCP/OCI sentaksı."""
        ...

    @abstractmethod
    def get_data_type_map(self) -> dict[str, str]:
        """Kaynak DB tip adı → hedef DB tip adı eşlemesi."""
        ...

    @abstractmethod
    def generate_ddl(self, schema: str, table: str, columns: list[dict]) -> str:
        """Hedef DB için CREATE TABLE DDL üretir."""
        ...

    @abstractmethod
    def get_pagination_query(self, base_sql: str, offset: int, limit: int) ->
str:
        """DB-specific pagination sentaksı."""
        ...

    # Sadece PostgresDialect (Enterprise) implement eder — diğerleri
NotImplementedError
    def generate_bulk_extract_query(self, inner_sql: str) -> str:
        raise NotImplementedError("Bu dialect COPY TO STDOUT desteklemiyor")
```

DBSession — Bağlantı Context Manager (W1)

```
# src/ffengine/db/session.py
class DBSession:
    """
    Driver-agnostic DB bağlantı context manager.
    Her dialect kendi connect() ve create_cursor() metodunu sağlar.
    """
    def __init__(self, connection_params: dict, dialect: BaseDialect):
        self.params  = connection_params
        self.dialect = dialect
        self.conn    = None

    def __enter__(self):
        self.conn = self.dialect.connect(self.params)
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type:
            self.conn.rollback()
        else:
```

```
            self.conn.commit()
        self.conn.close()

    def cursor(self, server_side: bool = False):
        return self.dialect.create_cursor(self.conn, server_side)
```

Sınıf Hiyerarşisi Özeti

Sınıf / Modül	Wave	Dosya	Implement Eden
BaseEngine	W1	core/engine_interface.py	PythonEngine (W3), CEngine (W6)
BaseDialect	W2	dialect/base.py	PostgresDialect, OracleDialect, MSSQLDialect (W2)
DBSession	W1	db/session.py	Tüm dialect'ler tarafından kullanılır
TypeMapper	W2	mapping/type_mapper.py	Sabit eşleme tablosu — statik metodlar
DDLGenerator	W2	mapping/ddl_generator.py	TypeMapper'ı kullanır
SchemaReader	W2	mapping/schema_reader.py	Dialect.get_table_schema() çağırır
SourceReader	W3	pipeline/source_reader.py	BaseDialect, DBSession kullanır
Streamer	W3	pipeline/streamer.py	SourceReader → TargetWriter arası buffer
TargetWriter	W3	pipeline/target_writer.py	BaseDialect.generate_bulk_insert_query()
Transformer	W3	pipeline/transformer.py	TABLE_RULES uygular, TypeMapper kullanır
ETLManager (Python)	W3	core/etl_manager.py	PythonEngine'in run() gövdesi
BindingResolver	W4	filters/binding_resolver.py	DBSession kullanır
Partitioner	W4	filters/partitioner.py	6 strateji sınıfı içerir
FFEngineOperator	W4	airflow/operator.py	BaseEngine.detect() çağırır
validate_config	W4	validation/config_validator.py	Standalone fonksiyon
mapping_generator	W5	tools/mapping_generator.py	SchemaReader + DBSession kullanır
FFEnvelope	W6	enterprise/queue_runtime/envelope.py	Enterprise Only
IngressQueue	W6	enterprise/queue_runtime/ingress_queue.py	Enterprise Only
EgressQueue	W6	enterprise/queue_runtime/egress_queue.py	Enterprise Only
CheckpointStore	W6	enterprise/queue_runtime/	Enterprise Only

Sınıf / Modül	Wave	Dosya	Implement Eden
		checkpoint_store.py	
DeliveryManager	W6	enterprise/queue_runtime/ delivery_manager.py	Enterprise Only
PGCopyBulk	W7	enterprise/bulk/pg_copy.py	Enterprise Only
MSSQLBCPBulk	W7	enterprise/bulk/mssql_bcp.py	Enterprise Only
OracleOCIBulk	W7	enterprise/bulk/oracle_oci.py	Enterprise Only
WorkerPool	W7	enterprise/bulk/worker_pool.py	Enterprise Only — N/M asimetrik
DLQPolicy	W8	enterprise/policy/dlq_policy.py	Enterprise Only
DeliveryPolicy	W8	enterprise/policy/ delivery_policy.py	Enterprise Only
MultiLanePipeline	W8	enterprise/policy/multi_lane.py	Enterprise Only

03 — Wave 1: Foundation & Core Infrastructure

WAVE 1	**Foundation & Core Infrastructure** Ortak — Community + Enterprise	Q1 2026 / Sprint 1-2

Wave 1 Amacı: Projenin tüm bileşenlerinin üzerine inşa edileceği temel iskelet. Bu wave tamamlanmadan hiçbir başka wave başlatılamaz.

Ön Koşullar

Yok — bu ilk wave'dir.

Teslim Edilecekler

Dosya	Sorumluluk	Kritik Detay
pyproject.toml	Paket tanımı, bağımlılıklar, entry point'ler	psycopg3, python-oracledb, pyodbc, pyyaml, apache-airflow bağımlılıkları
src/ffengine/__init__.py	Paket girişi, versiyon sabiti	__version__ = '1.0.0' (Community) / '2.0.0' (Enterprise)
core/engine_interface.py	BaseEngine abstract + ETLResult dataclass + detect()	Engine-Swap kontratı — değiştirilmez
db/session.py	DBSession context manager	Tüm DB işlemleri bu üzerinden geçer
db/connection_factory.py	Sürücü tespiti (psycopg3/oracledb/pyodbc) , bağlantı üretimi	Airflow Connection ID → params dönüşümü
core/config.py	YAML yükleyici, şema doğrulama (pydantic veya dataclass)	ETLConfig, TaskConfig dataclass'ları
Dockerfile	apache/airflow:3.1.6-python3.12 base image	Linux, 8 CPU, 32GB RAM referans ortam
docker-compose.yml	Airflow + Postgres + MSSQL + Oracle servis tanımları	Yerel geliştirme ortamı
tests/unit/test_config.py	Config yükleme ve şema doğrulama testleri	Zorunlu alan eksikliği, tip hatası senaryoları
tests/unit/test_session.py	DBSession context manager testleri	commit/rollback, cursor mock testleri

Kritik Kod — ETLConfig Dataclass

```
# src/ffengine/core/config.py
from dataclasses import dataclass, field
```

```
from typing import Literal, Optional

@dataclass
class BindingConfig:
    from_source: Literal["source","target","literal","airflow_var"]
    sql:     Optional[str] = None
    default: Optional[str] = None

@dataclass
class PartitionConfig:
    enabled: bool = False
    column:  Optional[str] = None
    mode:
Literal["auto","percentile","hash_mod","explicit","distinct","full_scan"] =
"auto"
    parts:   int = 2

@dataclass
class TaskConfig:
    task_group_id:        str
    source_schema:        str
    source_table:         str
    target_schema:        str
    target_table:         str
    source_type:          Literal["table","view","sql","csv","script"] = "table"
    column_mapping_mode:  Literal["source","mapping_file"] = "source"
    sql_file:             Optional[str] = None
    load_method:          str = "create_if_not_exists_or_truncate"
    where:                Optional[str] = None
    order_by:             Optional[str] = None
    bindings:             dict[str, BindingConfig] = field(default_factory=dict)
    passthrough_full:     bool = True
    passthrough_format:   Literal["binary","text","csv"] = "binary"
    pipe_queue_max:       int = 8
    batch_size:           int = 10000
    extraction_method:    Literal["copy_binary","cursor","auto"] = "auto"
    reader_workers:       int = 3
    writer_workers:       int = 5
    partitioning:         PartitionConfig =
field(default_factory=PartitionConfig)
    tags:                 list[str] = field(default_factory=list)

@dataclass
class ETLConfig:
    source_db_var: str
    target_db_var: str
    etl_tasks:     list[TaskConfig]
```

Wave 1 Tamamlanma Kriterleri

- pyproject.toml kurulumu ve pip install -e . başarılı
- DBSession unit testleri geçiyor
- ETLConfig YAML yükleme testleri geçiyor
- Docker compose ayaklanıyor (airflow, postgres, mssql, oracle)

04 — Wave 2: Dialect & TypeMapper

WAVE 2	**Dialect & TypeMapper** Ortak — Community + Enterprise	Q1 2026 / Sprint 2-3

Wave 2 Amacı: Tüm veritabanı bağımsızlığını sağlayan ortak katman. Bu wave'de üretilen sınıflar hem Community hem Enterprise tarafından değiştirilmeden kullanılır.

Ön Koşullar

Wave 1 tamamlanmış ve testler geçiyor olmalı.

Teslim Edilecekler

Dosya	Sorumluluk	Kritik Detay
dialect/base.py	BaseDialect abstract class	Tüm metodlar @abstractmethod — implementasyon boş bırakılmaz
dialect/postgres.py	PostgresDialect	generate_bulk_extract_query() eklenir (Enterprise'da aktif olur — W7)
dialect/oracle.py	OracleDialect	ALL_TAB_COLUMNS üzerinden metadata; 38 basamak precision dikkat
dialect/mssql.py	MSSQLDialect	sys.columns üzerinden metadata; pyodbc / UnixODBC
dialect/registry.py	Dialect seçici	'postgresql'→PostgresDialect, 'oracle'→OracleDialect, 'mssql'→MSSQLDialect
mapping/type_mapper.py	TypeMapper (statik eşleme tablosu)	Oracle→PG, Oracle→MSSQL, MSSQL→PG çapraz eşleme
mapping/ddl_generator.py	DDLGenerator	TypeMapper kullanarak hedef DB'ye uygun CREATE TABLE üretir
mapping/schema_reader.py	SchemaReader / MetadataInspector	dialect.get_table_schema() sargısı; DDLGenerator'a besleme
tests/unit/test_dialects.py	Tüm dialect metodları için unit test	Her DB için: metadata, bulk_insert_query, pagination
tests/unit/test_type_mapper.py	TypeMapper eşleme doğruluğu	Finansal tipler (NUMERIC precision/scale) kritik

Kritik Kod — TypeMapper Eşleme Tablosu

```
# src/ffengine/mapping/type_mapper.py
class TypeMapper:
    # Oracle → PostgreSQL eşlemesi
    ORACLE_TO_PG = {
```

```
        "NUMBER":    lambda p,s: f"NUMERIC({p},{s})" if s else ("BIGINT" if
p<=18 else f"NUMERIC({p},0)"),
        "VARCHAR2":  lambda p,_: f"VARCHAR({p})",
        "CHAR":      lambda p,_: f"CHAR({p})",
        "DATE":      lambda _,__: "TIMESTAMP",
        "TIMESTAMP": lambda p,_: f"TIMESTAMP({p or 6})",
        "CLOB":      lambda _,__: "TEXT",
        "BLOB":      lambda _,__: "BYTEA",
        "NVARCHAR2": lambda p,_: f"VARCHAR({p})",
    }
    # Oracle → MSSQL eşlemesi
    ORACLE_TO_MSSQL = {
        "NUMBER":    lambda p,s: f"DECIMAL({p},{s})" if s else ("BIGINT" if
p<=18 else f"DECIMAL({p},0)"),
        "VARCHAR2":  lambda p,_: f"NVARCHAR({p})",
        "DATE":      lambda _,__: "DATETIME2",
        "TIMESTAMP": lambda p,_: f"DATETIME2({p or 7})",
        "CLOB":      lambda _,__: "NVARCHAR(MAX)",
        "BLOB":      lambda _,__: "VARBINARY(MAX)",
    }
    # PG → MSSQL, MSSQL → PG ... (diğer çapraz eşlemeler)

    @classmethod
    def map(cls, source_db: str, target_db: str, source_type: str,
            precision: int = None, scale: int = None) -> str:
        key = f"{source_db.upper()}_TO_{target_db.upper()}"
        mapping = getattr(cls, key, {})
        base_type = source_type.upper().split("(")[0]
        fn = mapping.get(base_type)
        if fn:
            return fn(precision, scale)
        return source_type  # bilinmeyen tip → aynen geç
```

Wave 2 Tamamlanma Kriterleri

- Tüm dialect unit testleri geçiyor
- TypeMapper: NUMERIC(16,0) → BIGINT doğru eşleme
- TypeMapper: NUMERIC(38,10) → NUMERIC(38,10) precision kaybı yok
- DDLGenerator integration testi: Oracle tablosundan PG CREATE TABLE doğru üretiliyor
- Yeni dialect ekleme süresi < 1 saat (BaseDialect'ten türetme + registry kaydı)

05 — Wave 3: Community Engine (Python Core)

WAVE 3	**Community Engine — Python Core** Community	Q1 2026 / Sprint 3-4

Wave 3 Amacı: ffengine Community'nin veri taşıma motoru. fetchmany + executemany döngüsü, yield-based Streamer ve ETLManager (PythonEngine). Bu wave sonunda PG→PG tam aktarım çalışmalıdır.

Ön Koşullar

Wave 1 + Wave 2 tamamlanmış, tüm testler geçiyor.

Teslim Edilecekler

Dosya	Sorumluluk	Kritik Detay
pipeline/source_reader.py	SourceReader	fetchmany(batch_size) ile server-side cursor; OOM engelleme
pipeline/streamer.py	Streamer (Buffer Engine)	yield generator; pipe_queue_max backpressure; Producer-Consumer pattern
pipeline/target_writer.py	TargetWriter	executemany (standart DBAPI); load_method uygulaması
pipeline/transformer.py	Transformer + TABLE_RULES	strip, parse_dates, to_numeric, empty_to_null, add_etl_date
core/etl_manager.py	PythonEngine (BaseEngine impl.)	SourceReader→Streamer→TargetWriter koordinasyonu; XCom metrik üretimi
core/progress_tracker.py	ProgressTracker	rows/sec hesaplama; tahmini kalan süre; Airflow log basımı
core/error_handler.py	ErrorHandler	extract/transform/load aşama etiketi; chunk rollback
tests/unit/test_source_reader.py	SourceReader unit testleri	fetchmany mock; chunk boyutu doğrulama
tests/unit/test_streamer.py	Streamer unit testleri	backpressure throttle testi; yield sırası
tests/unit/test_etl_manager.py	ETLManager entegrasyon testi	mock DB ile tam ETL döngüsü
tests/integration/test_pg_to_pg.py	PG→PG tam aktarım	100K satır; RAM < 500MB; 3 dk altında

Kritik Kod — Streamer (Buffer Engine)

```
# src/ffengine/pipeline/streamer.py
import time
from queue import Queue
```

```
class Streamer:
    def __init__(self, pipe_queue_max: int = 8):
        self.pipe_queue_max = pipe_queue_max
        self.buffer = Queue(maxsize=pipe_queue_max)

    def stream(self, source_reader, target_writer, type_mapper,
batch_size=10000):
        """
        Community: Her chunk'ı executemany (standart DBAPI) ile hedefe yazar.
        Bu Streamer yalnızca Community kullanır.
        Enterprise için IngressQueue / EgressQueue (W6) kullanılır.
        """
        def producer():
            for chunk in source_reader.fetch_chunks(batch_size):
                yield chunk  # belleğe biriktirmeden doğrudan pipe

        for chunk in producer():
            typed_chunk = type_mapper.cast_chunk(chunk)
            target_writer.write_batch(typed_chunk)
            self._apply_backpressure()

    def _apply_backpressure(self):
        if self.buffer.qsize() >= self.pipe_queue_max:
            time.sleep(0.01)  # throttle
```

Kritik Kod — PythonEngine (BaseEngine Implementasyonu)

```
# src/ffengine/core/etl_manager.py
from ffengine.core.engine_interface import BaseEngine, ETLResult

class PythonEngine(BaseEngine):
    """Community Query-Centric Python Engine."""

    def is_available(self) -> bool:
        return True  # Python Engine her zaman mevcuttur

    def run(self, config_path: str, task_group_id: str) -> ETLResult:
        config   = load_config(config_path)
        task_cfg = get_task(config, task_group_id)

        validate_config(task_cfg)          # W4'te implement edilir

        src_dialect = DialectRegistry.get(config.source_db_var)
        tgt_dialect = DialectRegistry.get(config.target_db_var)

        with DBSession(src_dialect) as src_sess,
DBSession(tgt_dialect) as tgt_sess:

            schema_reader = SchemaReader(src_sess, src_dialect)
            ddl_generator = DDLGenerator(schema_reader, tgt_dialect)

            # Faz 2: Hedef tabloyu hazırla
            tgt_writer = TargetWriter(tgt_sess, tgt_dialect,
task_cfg.load_method)
            tgt_writer.prepare(ddl_generator)

            # Faz 3: Veri aktarımı
            src_reader = SourceReader(src_sess, src_dialect, task_cfg)
            streamer   = Streamer(task_cfg.pipe_queue_max)
            tracker    = ProgressTracker()

            streamer.stream(src_reader, tgt_writer, TypeMapper)

        return ETLResult(
            rows=tracker.total_rows,
```

```
            duration_seconds=tracker.elapsed,
            throughput=tracker.rows_per_sec,
            partitions_completed=1,
            errors=[]
        )
```

Wave 3 Tamamlanma Kriterleri

- PG→PG integration test: 1M satır aktarımı başarılı
- RAM kullanımı 500MB altında (1M satır için)
- chunk rollback: yazma hatasında transaction geri alınıyor
- ProgressTracker: rows/sec Airflow log'a basılıyor
- BaseEngine.is_available() = True (PythonEngine için)

06 — Wave 4: Airflow Integration & ETL Studio

WAVE 4	**Airflow Integration & ETL Studio** Ortak — Community + Enterprise	Q1 2026 / Sprint 4-5

Wave 4 Amacı: FFEngineOperator, Auto-DAG Generator, 3-Faz DAG Pattern, ETL Studio UI plugin ve validate_config(). Bu wave sonunda Airflow üzerinden tam E2E aktarım çalışmalıdır.

Ön Koşullar

Wave 1 + Wave 2 + Wave 3 tamamlanmış ve testler geçiyor.

Teslim Edilecekler

Dosya	Sorumluluk	Kritik Detay
airflow/operator.py	FFEngineOperator	BaseEngine.detect() çağırır; XCom push; automatic fallback
airflow/dag_generator.py	Auto-DAG Generator	YAML config → Airflow DAG dosyası üretir
airflow/dag_patterns.py	3-Faz DAG Pattern	plan_partitions → prepare_target → run_partition.expand()
filters/binding_resolver.py	BindingResolver	from: source/target/literal/airflow_var çözümleme
filters/partitioner.py	Partitioner (6 strateji)	AutoNumeric, Percentile, HashMod, Explicit, Distinct, FullScan
validation/config_validator.py	validate_config()	source_type, column_mapping_mode, zorunlu alanlar, dosya varlığı
db/pool_manager.py	Connection keep-alive	TCP keep-alive; uzun aktarımda timeout önleme
airflow/etl_studio/plugin.py	ETL Studio Airflow plugin	Admin > ETL Studio menüsü kaydı
airflow/etl_studio/views.py	ETL Studio görünümleri	Şema keşfi (50 limit), DAG oluşturma/güncelleme, tag, timeline
tests/unit/test_operator.py	FFEngineOperator testleri	engine=auto/community/enterprise; XCom mock
tests/unit/ test_validate_config.py	validate_config testleri	Her hata senaryosu için ayrı test
tests/unit/test_partitioner.py	Partitioner testleri	Tüm 6 mod için versiyon kontrolü
tests/integration/ test_airflow_dag.py	DAG oluşturma ve çalışma testi	plan→prepare→run sırası doğrulama

Kritik Kod — 3-Faz DAG Pattern

```
# src/ffengine/airflow/dag_patterns.py
from airflow.decorators import task, task_group
from airflow.operators.trigger_dagrun import TriggerDagRunOperator

def build_etl_task_group(etl_task_cfg: dict, dag):
    """
    Her etl_task için 3-faz TaskGroup oluşturur.
    plan_partitions → prepare_target → run_partition.expand()
    """
    with task_group(group_id=etl_task_cfg["task_group_id"], dag=dag) as tg:

        @task
        def plan_partitions(task_cfg):
            from ffengine.filters.binding_resolver import BindingResolver
            from ffengine.filters.partitioner import Partitioner
            resolved = BindingResolver(task_cfg).resolve()
            return Partitioner(task_cfg).generate(resolved)
            # XCom → [{"where": "id BETWEEN 0 AND 50000"}, ...]

        @task
        def prepare_target(task_cfg):
            from ffengine.pipeline.target_writer import TargetWriter
            TargetWriter.prepare_table(task_cfg)

        @task
        def run_partition(partition_spec, task_cfg):
            from ffengine.core.engine_interface import BaseEngine
            engine = BaseEngine.detect(task_cfg.get("engine","auto"))
            return engine.run_partition(task_cfg, partition_spec)

        specs = plan_partitions(etl_task_cfg)
        prep  = prepare_target(etl_task_cfg)
        # Dynamic Task Mapping — her spec ayrı worker'da çalışır
        specs >> prep >> run_partition.expand(partition_spec=specs)
    return tg
```

Kritik Kod — validate_config()

```
# src/ffengine/validation/config_validator.py
import os
from ffengine.core.config import TaskConfig

VALID_SOURCE_TYPES   = {"table","view","sql","csv","script"}
VALID_MAPPING_MODES  = {"source","mapping_file"}
VALID_LOAD_METHODS   = {
    "create_if_not_exists_or_truncate","append","replace",
    "upsert","delete_from_table","drop_if_exists_and_create","script"
}

def validate_config(task: TaskConfig) -> None:
    """Çalışma öncesi YAML konfigürasyonu doğrular. Hatalarda ValueError
fırlatır."""
    assert task.task_group_id, "task_group_id zorunludur"
    assert task.source_schema,  "source_schema zorunludur"
    assert task.target_schema,  "target_schema zorunludur"
    assert task.target_table,   "target_table zorunludur"

    if task.source_type not in VALID_SOURCE_TYPES:
        raise ValueError(f"Geçersiz source_type: {task.source_type}")
    if task.column_mapping_mode not in VALID_MAPPING_MODES:
        raise ValueError(f"Geçersiz column_mapping_mode:
{task.column_mapping_mode}")
    if task.load_method not in VALID_LOAD_METHODS:
        raise ValueError(f"Geçersiz load_method: {task.load_method}")
```

```
    if task.column_mapping_mode == "mapping_file":
        mapping_path =
f"mappings/{task.source_schema}/{task.task_group_id}.yaml"
        if not os.path.exists(mapping_path):
            raise FileNotFoundError(f"Mapping dosyası bulunamadı:
{mapping_path}")

    if task.source_type == "sql" and task.sql_file:
        if not os.path.exists(task.sql_file):
            raise FileNotFoundError(f"SQL dosyası bulunamadı: {task.sql_file}")
```

Wave 4 Tamamlanma Kriterleri

- FFEngineOperator engine='auto' — PythonEngine devreye giriyor
- Auto-DAG Generator: YAML'dan geçerli DAG üretiyor
- 3-Faz DAG: plan→prepare→run sırası Airflow'da doğru çalışıyor
- validate_config: tüm hata senaryoları yakalanıyor
- ETL Studio UI: şema keşfi ve DAG oluşturma çalışıyor
- TriggerDagRunOperator: Level1→Level2 kademeli tetikleme çalışıyor

07 — Wave 5: Community GA — Operasyonel Araçlar

WAVE 5	**Community GA — Operasyonel Araçlar & Release** Community	Q1 2026 / Sprint 5-6

Wave 5 Amacı: Community versiyonunun production'a çıkışı. mapping_generator CLI, email notification, Dry-Run modu, PyPI paketi. Bu wave sonunda ffengine community tam olarak kullanıma hazırdır.

Ön Koşullar

Wave 1–4 tamamlanmış ve testler geçiyor.

Teslim Edilecekler

Dosya	Sorumluluk	Kritik Detay
tools/mapping_generator.py	mapping_generator CLI	Kaynak DB'ye bağlanarak YAML mapping otomatik üretir; _v1, _v2 versiyonlama
tools/send_email_custom.py	Email Notification	on_failure_callback; Türkçe HTML; DAG ID, Task ID, süre, hostname, stacktrace
tools/debug_utils.py	debugpy Remote Debug	wait_for_debug(port=5678); VS Code uzak debug desteği
pipeline/dry_run.py	Dry-Run Modu	--dry-run: DDL üretimi + bağlantı testi; veri aktarımı yapılmaz
tests/integration/ test_community_e2e.py	Community tam E2E test	mapping_generator→config→dag→run→verify tam döngü
RELEASE: PyPI Paketi	ffengine community PyPI yayını	pip install ffengine
docs/community_quickstart.md	Kurulum ve ilk pipeline kılavuzu	10 dakikada ilk pipeline

Kritik Kod — mapping_generator CLI

```
# src/ffengine/tools/mapping_generator.py
"""
Kullanım:
  python -m ffengine.tools.mapping_generator \
    --conn-id src_db \
    --schema ocn_iss \
    --table crd_account_balance_used_hist \
    --output-dir projects/webhook/whk/level1/src_to_stg/mappings/
"""
import argparse, yaml, datetime, os
from ffengine.mapping.schema_reader import SchemaReader
from ffengine.mapping.type_mapper   import TypeMapper
```

```
def generate(conn_id, schema, table, target_db="postgresql",
             sql=None, output_dir="mappings/"):
    columns = SchemaReader.from_conn(conn_id).read(schema, table, sql)
    mapping = {
        "version": "1.0",
        "mapping_type": "column",
        "generated_at": datetime.datetime.utcnow().isoformat() + "Z",
        "source": {"schema": schema, "table": table, **({"sql": sql} if sql else
{})},
        "columns": [
            {"source": c["name"], "target": c["name"],
             "type": TypeMapper.map("oracle", target_db, c["type"],
                                    c.get("precision"), c.get("scale"))}
            for c in columns
        ]
    }
    os.makedirs(output_dir, exist_ok=True)
    fname = f"{table}_to_{schema}_{table}_v1.yaml"
    fpath = os.path.join(output_dir, fname)
    with open(fpath, "w") as f:
        yaml.dump(mapping, f, allow_unicode=True, sort_keys=False)
    return fpath
```

Community Release Kontrol Listesi

Kontrol	Beklenen Sonuç	Test
10M satır PG→PG aktarım	< 3 dakika, < 500MB RAM	test_community_e2e.py::test_10m_pg_to_pg
10M satır Oracle→PG aktarım	< 5 dakika, < 500MB RAM	test_community_e2e.py::test_oracle_to_pg
10M satır PG→MSSQL aktarım	< 5 dakika, < 500MB RAM	test_community_e2e.py::test_pg_to_mssql
validate_config tüm hata senaryoları	ValueError / FileNotFoundError	test_validate_config.py
mapping_generator CLI	Geçerli YAML üretimi, v1 versiyonlama	test_mapping_generator.py
Dry-Run modu	DDL üretiliyor, veri aktarımı yok	test_dry_run.py
Email notification	HTML email gönderimi (mock SMTP)	test_send_email.py
ETL Studio UI	DAG oluşturma, tag yönetimi, timeline	test_etl_studio.py
PyPI kurulumu	pip install ffengine çalışıyor	CI/CD release pipeline

08 — Wave 6: Enterprise Queue Runtime

WAVE 6	**Enterprise Queue Runtime (C Binding)** Enterprise	Q2 2026 / Sprint 7-8

Wave 6 Amacı: Queue-Aware mimarinin Python tarafındaki köprüsü. FFEnvelope, IngressQueue, EgressQueue, CheckpointStore ve DeliveryManager implementasyonu. C Engine Python binding iskelet kodu.

Ön Koşullar

Wave 1–5 tamamlanmış (Community GA canlıda). Enterprise C Engine C kütüphanesi derlenmiş ve test edilmiş.

Teslim Edilecekler

Dosya	Sorumluluk	Kritik Detay
enterprise/engine.py	CEngine (BaseEngine impl.)	is_available(): C lib yüklü mü? run(): C Engine'i tetikler
enterprise/queue_runtime/envelope.py	FFEnvelope dataclass	header(metadata, partition_key, sequence) + payload(data chunk)
enterprise/queue_runtime/ingress_queue.py	IngressQueue	Thread-safe Queue; maxsize=pipe_queue_max; FFEnvelope wrapper
enterprise/queue_runtime/egress_queue.py	EgressQueue	Delivery policy uygulama; ordered / at-least-once semantik
enterprise/queue_runtime/checkpoint_store.py	CheckpointStore	Source offset + target write ack; JSON veya SQLite; resume yeteneği
enterprise/queue_runtime/delivery_manager.py	DeliveryManager	Retry sayacı, delivery policy uygulama, poison message tespiti
enterprise/queue_runtime/backpressure.py	BackpressureController	Queue doluluk oranı → producer throttle
tests/unit/test_ingress_queue.py	IngressQueue unit testleri	backpressure, envelope wrap/unwrap, thread safety
tests/unit/test_checkpoint_store.py	CheckpointStore unit testleri	offset kaydetme, resume senaryosu, corruption recovery
tests/integration/test_enterprise_queue.py	Queue Runtime E2E testi	Source→IngressQ→EgressQ→Target tam döngü

Kritik Kod — FFEnvelope ve IngressQueue

```
# src/ffengine/enterprise/queue_runtime/envelope.py
from dataclasses import dataclass, field
from typing import Any
import uuid, time
```

```
@dataclass
class FFEnvelopeHeader:
    envelope_id:   str  = field(default_factory=lambda: str(uuid.uuid4()))
    partition_key: str  = ""
    sequence:      int  = 0
    created_at:    float = field(default_factory=time.time)
    source_offset: Any  = None  # PK / offset değeri

@dataclass
class FFEnvelope:
    header:  FFEnvelopeHeader
    payload: list          # data chunk (satır listesi)
    ack:     bool  = False # True: başarıyla işlendi
    nack:    bool  = False # True: hata — retry veya DLQ

# src/ffengine/enterprise/queue_runtime/ingress_queue.py
import threading
from queue import Queue, Full
from ffengine.enterprise.queue_runtime.envelope import FFEnvelope,
FFEnvelopeHeader

class IngressQueue:
    """Source Producer → Transformer arası thread-safe kuyruk."""
    SENTINEL = None  # akış sonu sinyali

    def __init__(self, maxsize: int = 8):
        self._q = Queue(maxsize=maxsize)
        self._lock = threading.Lock()

    def put(self, chunk: list, partition_key: str = "", offset = None):
        envelope = FFEnvelope(
            header=FFEnvelopeHeader(partition_key=partition_key,
source_offset=offset),
            payload=chunk
        )
        self._q.put(envelope)   # kuyruk doluysa otomatik block (backpressure)

    def get(self) -> FFEnvelope | None:
        return self._q.get()

    def put_sentinel(self):
        self._q.put(self.SENTINEL)
```

Kritik Kod — CheckpointStore

```
# src/ffengine/enterprise/queue_runtime/checkpoint_store.py
import json, os
from threading import Lock

class CheckpointStore:
    """
    Source offset ve target write ack takibi.
    Aktarım kesilirse bu store üzerinden kaldığı yerden devam edilir.
    """
    def __init__(self, checkpoint_path: str):
        self.path = checkpoint_path
        self._lock = Lock()
        self._state = self._load()

    def _load(self) -> dict:
        if os.path.exists(self.path):
            with open(self.path) as f:
                return json.load(f)
        return {"last_source_offset": None, "last_ack_sequence": 0, "completed":
False}
```

```
    def save_offset(self, offset):
        with self._lock:
            self._state["last_source_offset"] = offset
            self._flush()

    def ack(self, sequence: int):
        with self._lock:
            self._state["last_ack_sequence"] = sequence
            self._flush()

    def get_resume_offset(self):
        return self._state.get("last_source_offset")

    def _flush(self):
        with open(self.path, "w") as f:
            json.dump(self._state, f)
```

Wave 6 Tamamlanma Kriterleri

- CEngine.is_available(): C lib yüklüyse True döndürüyor
- FFEngineOperator engine='auto' → CEngine seçiyor
- IngressQueue thread safety testi geçiyor
- CheckpointStore resume testi: kesilen aktarım kaldığı yerden devam ediyor
- EgressQueue at-least-once delivery testi geçiyor

09 — Wave 7: Enterprise Native Bulk API

WAVE 7	**Enterprise Native Bulk API** Enterprise	Q2 2026 / Sprint 8-9

Wave 7 Amacı: COPY TO STDOUT / FROM STDIN (PG binary), BCP (MSSQL), OCI_BATCH (Oracle) ve N Reader / M Worker asimetrik worker pool implementasyonu. Enterprise throughput hedeflerine ulaşılır.

Ön Koşullar

Wave 6 tamamlanmış, Queue Runtime testleri geçiyor.

Teslim Edilecekler

Dosya	Sorumluluk	Kritik Detay
enterprise/bulk/pg_copy.py	PGCopyBulk	COPY TO STDOUT (kaynak PG) + COPY FROM STDIN (hedef PG); psycopg3 streaming API; binary format
enterprise/bulk/mssql_bcp.py	MSSQLBCPBulk	fast_executemany=True (pyodbc); BCP protocol; Enterprise Only
enterprise/bulk/oracle_oci.py	OracleOCIBulk	executemany + OCI_BATCH_ERRORS; hatalı satır izolasyonu
enterprise/bulk/worker_pool.py	WorkerPool (N/M Asimetrik)	reader_workers + writer_workers YAML'dan okunur; threading.Thread pool
dialect/postgres.py	PostgresDialect güncelleme	generate_bulk_extract_query() aktif hale gelir (W2'de tanımlandı, W7'de implement edilir)
tests/unit/test_pg_copy.py	PG COPY unit testleri	binary format doğrulama; sentinel propagation
tests/integration/ test_enterprise_bulk.py	Bulk API E2E testleri	10M satır PG→PG; throughput 500K+/sn hedefi

Kritik Kod — PG Binary COPY Queue Pipeline

```
# src/ffengine/enterprise/bulk/pg_copy.py
import threading, psycopg
from queue import Queue
from ffengine.enterprise.queue_runtime.ingress_queue import IngressQueue

SENTINEL = None

class PGCopyPipeline:
    """
    Enterprise Only: COPY TO STDOUT → IngressQueue → COPY FROM STDIN
    PG→PG senaryosunda uçtan uca binary pipe — sıfır Python nesne dönüşümü.
    """
    def __init__(self, config):
```

```
        self.reader_workers = config.reader_workers  # default: 3
        self.writer_workers = config.writer_workers  # default: 5
        self.pipe_queue_max = config.pipe_queue_max  # default: 8
        self.queue = Queue(maxsize=self.pipe_queue_max)

    def _reader_worker(self, partition_sql: str, source_dsn: str):
        with psycopg.connect(source_dsn) as conn:
            with conn.cursor().copy(
                f"COPY ({partition_sql}) TO STDOUT (FORMAT BINARY)"
            ) as copy:
                for data in copy:
                    self.queue.put(data)  # binary blok → queue (otomatik
backpressure)
        self.queue.put(SENTINEL)  # okuma bitti sinyali

    def _writer_worker(self, target_dsn: str, target_table: str, columns: list,
                       checkpoint_store=None):
        with psycopg.connect(target_dsn) as conn:
            col_list = ",".join(columns)
            with conn.cursor().copy(
                f"COPY {target_table} ({col_list}) FROM STDIN (FORMAT BINARY)"
            ) as copy:
                while True:
                    data = self.queue.get()
                    if data is SENTINEL:
                        self.queue.put(SENTINEL)  # diğer writer'lara yay
                        break
                    copy.write(data)  # binary blok → hedef PG (Python nesne
YOK)
                    if checkpoint_store:
                        checkpoint_store.ack(id(data))

    def run(self, partition_sqls, source_dsn, target_dsn, table, columns,
checkpoint=None):
        readers = [threading.Thread(target=self._reader_worker, args=(sql,
source_dsn))
                   for sql in partition_sqls[:self.reader_workers]]
        writers = [threading.Thread(target=self._writer_worker,
                   args=(target_dsn, table, columns, checkpoint))
                   for _ in range(self.writer_workers)]
        for t in readers + writers: t.start()
        for t in readers + writers: t.join()
```

Wave 7 Tamamlanma Kriterleri

- 10M satır PG→PG: 500K+ satır/sn throughput
- RAM < 256MB (10M satır için)
- OCI_BATCH_ERRORS: hatalı satır izole ediliyor, bulk devam ediyor
- N=3 reader + M=5 writer asimetrik yapı çalışıyor
- Valgrind: C extension'da memory leak yok (CI/CD'de zorunlu)

10 — Wave 8: Enterprise GA — DLQ, Multi-Lane & Delivery Guarantee

WAVE 8	**Enterprise GA — DLQ, Multi-Lane & Delivery Guarantee** Enterprise	Q2 2026 / Sprint 9-10

Wave 8 Amacı: DLQ Policy, Multi-Lane Pipeline ve Delivery Guarantee Matrix'in tam implementasyonu. Enterprise versiyonunun production release'i.

Ön Koşullar

Wave 6 + Wave 7 tamamlanmış, Enterprise Queue Runtime ve Native Bulk API testleri geçiyor.

Teslim Edilecekler

Dosya	Sorumluluk	Kritik Detay
enterprise/policy/dlq_policy.py	DLQPolicy	Exponential backoff retry; max_retries; poison message → DLQ tablosuna yönlendirme
enterprise/policy/ retry_policy.py	RetryPolicy	Yeniden deneme sayacı; backoff hesaplama; DLQPolicy'e escalation
enterprise/policy/ delivery_policy.py	DeliveryPolicy	at-least-once (default); exactly-once koşul kontrolü; idempotency check
enterprise/policy/multi_lane.py	MultiLanePipeline	Bağımsız lane tanımları; lane bazlı checkpoint/pool/retry/ordering_key
tests/unit/test_dlq_policy.py	DLQ unit testleri	max retry aşımı, DLQ yönlendirme, poison message izolasyonu
tests/integration/ test_enterprise_e2e.py	Enterprise tam E2E	Delivery Guarantee Matrix senaryoları; Multi-Lane; DLQ
RELEASE: Enterprise Binary Wheel	ffengine enterprise private registry yayını	Binary wheel + RSA lisans modülü

Kritik Kod — DLQPolicy ve DeliveryPolicy

```
# src/ffengine/enterprise/policy/dlq_policy.py
import time
from dataclasses import dataclass

@dataclass
class DLQPolicy:
    max_retries:   int   = 3
    base_delay:    float = 1.0    # saniye
    max_delay:     float = 60.0   # saniye
    backoff_factor: float = 2.0

    def get_delay(self, attempt: int) -> float:
```

```
        delay = self.base_delay * (self.backoff_factor ** attempt)
        return min(delay, self.max_delay)

    def should_dlq(self, attempt: int) -> bool:
        return attempt >= self.max_retries

# src/ffengine/enterprise/policy/delivery_policy.py
from enum import Enum

class DeliverySemantics(Enum):
    AT_MOST_ONCE    = "at_most_once"     # checkpoint yok
    AT_LEAST_ONCE   = "at_least_once"    # varsayılan
    EXACTLY_ONCE    = "exactly_once"     # koşullu

class DeliveryPolicy:
    """
    Delivery Guarantee Matrix implementasyonu.
    Koşullar sağlanmıyorsa exactly-once'ı at-least-once'a düşürür.
    """
    def resolve_semantics(self, has_checkpoint: bool, has_idempotent_sink: bool,
                          has_ordering_key: bool) -> DeliverySemantics:
        if not has_checkpoint:
            return DeliverySemantics.AT_MOST_ONCE
        if has_idempotent_sink and has_checkpoint:
            # Exactly-once koşulları sağlandı
            return DeliverySemantics.EXACTLY_ONCE
        return DeliverySemantics.AT_LEAST_ONCE
```

Delivery Guarantee Matrix — Implementasyon Kontrol Tablosu

Kombinasyon	has_checkpoint	has_idempotent_sink	has_ordering_key	Sonuç Semantik
Queue + Idempotent + Ordering Key	True	True	True	EXACTLY_ONCE
Queue + Idempotent (no ordering)	True	True	False	EXACTLY_ONCE (sırasız)
Queue + no idempotent	True	False	Any	AT_LEAST_ONCE
Native Queue (AQ/SB) + idempotent	True	True	Any	EXACTLY_ONCE
Bulk Writer + upsert	True	True (upsert)	False	EXACTLY_ONCE (effectively)
Bulk Writer + truncate+insert	True	True (idempotent)	False	EXACTLY_ONCE (yeniden yüklenebilir)
Bulk Writer only (no upsert)	True	False	False	AT_LEAST_ONCE
No checkpoint	False	Any	Any	AT_MOST_ONCE

Enterprise Release Kontrol Listesi

Kontrol	Beklenen Sonuç	Test
10M satır PG→PG (Enterprise)	< 1 dakika, 500K+ satır/sn, < 256MB RAM	test_enterprise_e2e.py::test_10m_throughput
DLQ: 3 retry sonrası yönlendirme	Poison message DLQ tablosuna yazılıyor	test_dlq_policy.py::test_max_retries
Checkpoint resume	8M satırda kesilen aktarım 8M'den devam ediyor	test_enterprise_e2e.py::test_resume
DeliveryPolicy: exactly-once koşul kontrolü	Koşullar sağlanmıyorsa at-least-once seçiliyor	test_delivery_policy.py
Multi-Lane: 2 bağımsız lane	Lane bazlı checkpoint ve pool limiti	test_multi_lane.py
Valgrind: memory leak yok	0 leak (C extension)	CI/CD valgrind stage
FFEngineOperator fallback	C lib yoksa PythonEngine devreye giriyor	test_operator.py::test_fallback

11 — Wave Bağımlılık Grafiği ve Geçiş Kriterleri

Wave Bağımlılık Zinciri

```
  W1 Foundation
   └── W2 Dialect & TypeMapper
        └── W3 Community Engine
             └── W4 Airflow Integration
                  └── W5 Community GA ◄──── COMMUNITY RELEASE (Q1 2026)
                       └── W6 Enterprise Queue Runtime
                            └── W7 Enterprise Native Bulk API
                                 └── W8 Enterprise GA ◄──── ENTERPRISE RELEASE
(Q2 2026)
```

Her Wave İçin Geçiş Kriteri Özeti

Wave	Geçiş Kriteri	Blocker mı?
W1 → W2	DBSession + ETLConfig testleri geçiyor; Docker compose çalışıyor	Evet
W2 → W3	Tüm dialect unit testleri geçiyor; TypeMapper precision testi geçiyor	Evet
W3 → W4	PG→PG integration testi geçiyor; PythonEngine.is_available()=True	Evet
W4 → W5	3-Faz DAG Airflow'da çalışıyor; validate_config tüm senaryoları yakalıyor	Evet
W5 → W6	Community E2E testi geçiyor; PyPI paketi yayında; tüm W1-W5 testleri yeşil	Evet
W6 → W7	Queue Runtime E2E testi geçiyor; CheckpointStore resume testi geçiyor	Evet
W7 → W8	10M satır throughput hedefi sağlandı; Valgrind 0 leak; Bulk API E2E geçiyor	Evet
W8 Release	Enterprise E2E testi geçiyor; Delivery Guarantee Matrix doğrulandı; DLQ testi geçiyor	Evet

CI/CD Pipeline Akışı

```
# Her wave için CI/CD kontrol adımları:

Wave W1-W5 (Community):
  1. lint (ruff / flake8)
  2. unit tests (pytest tests/unit/)
  3. integration tests (pytest tests/integration/test_pg_to_pg.py ...)
  4. type check (mypy src/)
  5. coverage >= %80
  6. [W5 only] pip build + twine upload (PyPI)

Wave W6-W8 (Enterprise):
  1. lint + unit tests + integration tests (yukarıdaki gibi)
  2. valgrind --leak-check=full (C extension)
  3. enterprise throughput benchmark (500K satır/sn hedefi)
  4. delivery guarantee matrix scenario tests
  5. [W8 only] binary wheel build + private registry upload
```

12 — Agent Kodlama Kuralları ve Anti-Pattern'ler

Kesin Kurallar (Bunları İhlal Etme)

Kural	Açıklama	Neden?
Community Engine'e COPY/BCP/OCI ekleme	TargetWriter.write_batch() daima executemany kullanır	Yanlış beklenti; performans tutarsızlığı
Community'ye Checkpoint/DLQ ekleme	CheckpointStore ve DLQPolicy sadece enterprise/ altında	Versiyon sınırı ihlali
BaseEngine interface'ini bozma	ETLResult döndürme imzası değiştirilmez	Automatic fallback kırılır
BaseDialect metodlarını atla	Tüm @abstractmethod metodlar implement edilmeli	Yeni dialect eksik kalır
extraction_method: copy_binary Community'de aktif etme	Community'de bu mod çalışmaz; cursor kullanılmalı	Runtime hatası
Wave sıralamasını atla	W3'ü W2 tamamlanmadan başlatma	Bağımlılık hatası
exactly-once koşulsuz kullan	DeliveryPolicy.resolve_semantics() çağrılmadan exactly-once iddia etme	Yanlış delivery guarantee
LOB tiplerini streaming'e sok	CLOB/BLOB için chunking stratejisi tanımlanmalı	OOM riski
Valgrind'siz Enterprise release	W7/W8'de Valgrind CI adımı atlanmaz	Worker crash riski

Doğru Kodlama Desenleri

```
# ✓ DOĞRU: Engine tespiti her zaman BaseEngine.detect() üzerinden
engine = BaseEngine.detect("auto")  # automatic fallback

# ✗ YANLIŞ: Direkt PythonEngine veya CEngine kullanma
engine = PythonEngine()  # fallback mekanizması devre dışı

# ✓ DOĞRU: Dialect seçimi her zaman DialectRegistry üzerinden
dialect = DialectRegistry.get(conn_id)

# ✗ YANLIŞ: Direkt dialect örneği oluşturma
dialect = PostgresDialect()  # conn_id bilgisi kaybolur

# ✓ DOĞRU: TypeMapper.map() ile tip dönüşümü
pg_type = TypeMapper.map("oracle", "postgresql", "NUMBER", precision=16,
scale=0)
# → "BIGINT"

# ✗ YANLIŞ: Hardcode tip yazma
pg_type = "BIGINT"  # precision/scale bilgisi yok; başka tiplerde patlayabilir

# ✓ DOĞRU: Community Streamer (W3)
streamer = Streamer(pipe_queue_max=config.pipe_queue_max)
streamer.stream(source_reader, target_writer, TypeMapper)

# ✗ YANLIŞ: Community Streamer'a COPY komutu ekleme
# Community Streamer sadece yield + executemany kullanır
```

```
# ✓ DOĞRU: exactly-once koşul kontrolü (Enterprise)
semantics = delivery_policy.resolve_semantics(
    has_checkpoint=True, has_idempotent_sink=True, has_ordering_key=True
)  # → EXACTLY_ONCE

# ✗ YANLIŞ: Koşulsuz exactly-once iddiası
delivery = "exactly_once"  # koşullar kontrol edilmedi
```

Test Yazma Kuralları

- Her yeni dialect için: get_table_schema, generate_bulk_insert_query, generate_ddl testleri yazılmalı
- TypeMapper testi: NUMERIC(38,10) → NUMERIC(38,10) — precision kaybı 0
- SourceReader testi: fetchmany chunk'ları doğru boyutta, RAM sabit
- CheckpointStore testi: resume sonrası satır duplicate yok
- DLQPolicy testi: max_retries aşımında DLQ tablosuna yönlendirme
- DeliveryPolicy testi: koşullar eksikse exactly-once at-least-once'a düşüyor
- Integration testleri gerçek DB bağlantısı gerektirir — docker-compose up

13 — Gelecek Wave'ler (2027+)

Aşağıdaki wave'ler mevcut fazda kapsam dışıdır. Implementasyon başlatılmamalıdır; yalnızca placeholder interface tanımlanabilir.

Wave	İsim	Kapsam	Durum	Notlar
W9	MySQL Dialect	Ortak	Ertelenmiş	BaseDialect'ten türetme; registry kaydı; type mapping eklenecek
W10	Kafka / RabbitMQ Broker Entegrasyonu	Enterprise	Ertelenmiş	Source/Target Adapter olarak eklenir; W6 Queue Runtime üzerine inşa edilir
W11	Snowflake C-Connector	Enterprise	Ertelenmiş	Native Snowflake bulk API; W7 bulk/ dizinine eklenir
W12	BigQuery C-Connector	Enterprise	Ertelenmiş	Google Cloud native API; W7 bulk/ dizinine eklenir
W13	Hybrid Cloud Streaming	Enterprise	Ertelenmiş	On-premise Oracle/MSSQL → Cloud DB; W8 Multi-Lane üzerine inşa edilir
W14	Oracle AQ / TxEventQ Entegrasyonu	Enterprise	Ertelenmiş	W6 SourceAdapter olarak eklenir; python-oracledb AQ API
W15	MSSQL Service Broker Entegrasyonu	Enterprise	Ertelenmiş	W6 SourceAdapter olarak eklenir; pyodbc Service Broker API

Agent Uyarısı: W9–W15 için kod üretilmemelidir. Yalnızca şu kalıp kullanılabilir: raise NotImplementedError('Gelecek faz — Wave N planlanıyor'). Implementasyon başlatmak mevcut wave'lerle çakışmaya yol açabilir.